

# 代码中 Strang Splitting 离散公式整理

## 1. 总体说明

代码中的深度方向  $x$  是 marching 变量。记一个完整深度步长为  $\Delta x$ ，代码变量为 `grid.dt`。主粒子和二次粒子都使用同一个 Strang splitting 框架：

$$\psi^{n+1} = \mathcal{E}_{\Delta x/2} \mathcal{T}_{\Delta x/2} \mathcal{A}_{\Delta x} \mathcal{T}_{\Delta x/2} \mathcal{E}_{\Delta x/2} \psi^n. \quad (1)$$

对应代码调用顺序：

$$\text{energy half} \rightarrow \text{transport half} \rightarrow \text{angle full} \rightarrow \text{transport half} \rightarrow \text{energy half}. \quad (2)$$

论文公式对应关系如下：

|                         |                                   |                               |     |
|-------------------------|-----------------------------------|-------------------------------|-----|
| $\mathcal{E}$           | energy/DG/CN step                 | Eq. (15)                      |     |
| $\mathcal{T}$           | lateral ( $y, z$ ) transport step | Eq. (18), Eq. (22)            |     |
| $\mathcal{A}$           | angular diffusion step            | Eq. (19), Eq. (20), Eq. (21)  | (3) |
| primary/secondary split | $\psi = \psi_p + \psi_s$          | Eq. (9), Eq. (10a), Eq. (10b) |     |

当前代码中 CN 使用位置为：

| 子步            | 论文公式               | 是否使用 CN | 代码实现方式                       |
|---------------|--------------------|---------|------------------------------|
| $\mathcal{E}$ | Eq. (15)           | 是       | DG/CN 半隐式能量递推                |
| $\mathcal{T}$ | Eq. (18), Eq. (22) | 否       | 显式 MUSCL predictor-corrector |
| $\mathcal{A}$ | Eq. (19)–(21)      | 是       | sine/DST 对角化后的 CN 谱更新        |

 (4)

注意：论文 Remark 3 说明 Eq. (22) 的 depth discretization 也可以采用 CN，并可显式处理 slope；但当前代码没有按这个隐式/CN 形式实现 transport，而是使用显式二阶 predictor-corrector。

## 2. 代码变量与论文字母

| 代码变量       | 论文/数学符号                        | 含义                                       |
|------------|--------------------------------|------------------------------------------|
| F          | $\psi_{i,j,g,m}^1$             | DG 能量第一个系数, 或分布主系数                       |
| f_F        | $\psi_{i,j,g,m}^2$             | DG 能量第二个系数/斜率系数                          |
| F1, f_F1   | $\psi_s^1, \psi_s^2$           | 二次粒子对应两个 DG 系数                           |
| dt         | $\Delta x$                     | 深度步长, transport/energy 半步传入 $\Delta x/2$ |
| dg         | $\Delta E$                     | 能量组宽度                                    |
| dy, dz     | $\Delta y, \Delta z$           | 横向网格距                                    |
| du, dv     | $\Delta u, \Delta v$           | 角变量网格距                                   |
| Omend      | $(\Omega_{m,y}, \Omega_{m,z})$ | 横向角方向                                    |
| S_s[g]     | $S_g$                          | stopping power                           |
| T_c[g]     | $T_g$                          | straggling 系数                            |
| sigma_c[g] | $\sigma_{c,g}$                 | catastrophic loss 截面                     |
| sig_trg[g] | $\Sigma_{tr,g}$                | angular transport cross section          |
| density    | $\rho$                         | 材料密度因子                                   |

## 3. 主粒子 Strang Splitting

主粒子对应论文 Eq. (9), Eq. (10a)。代码中每一步为

$$\begin{aligned}
\psi_p^{(1)} &= \mathcal{E}_{\Delta x/2}(\psi_p^n), \\
\psi_p^{(2)} &= \mathcal{T}_{\Delta x/2}(\psi_p^{(1)}), \\
\psi_p^{(3)} &= \mathcal{A}_{\Delta x}(\psi_p^{(2)}), \\
\psi_p^{(4)} &= \mathcal{T}_{\Delta x/2}(\psi_p^{(3)}), \\
\psi_p^{n+1} &= \mathcal{E}_{\Delta x/2}(\psi_p^{(4)}).
\end{aligned} \tag{5}$$

代码对  $\psi^1$  和  $\psi^2$  两个 DG 系数都执行同样的 transport 和 angle 子步。

## 4. 二次粒子 Strang Splitting

二次粒子对应论文 Eq. (9), Eq. (10b)。代码结构同主粒子, 但 energy step 使用由主粒子构造的 source  $Q_p$ :

$$\begin{aligned}
\psi_s^{(1)} &= \mathcal{E}_{\Delta x/2}^{Q_p}(\psi_s^n; \psi_p), \\
\psi_s^{(2)} &= \mathcal{T}_{\Delta x/2}(\psi_s^{(1)}), \\
\psi_s^{(3)} &= \mathcal{A}_{\Delta x}(\psi_s^{(2)}), \\
\psi_s^{(4)} &= \mathcal{T}_{\Delta x/2}(\psi_s^{(3)}), \\
\psi_s^{n+1} &= \mathcal{E}_{\Delta x/2}^{Q_p}(\psi_s^{(4)}; \psi_p).
\end{aligned} \tag{6}$$

二次源离散求和在代码中为

$$Q_{g,m}^1(i, j) = \Delta E \sum_{g'} \sum_{m'} \left[ K_{g,g'}^{E,1} K_{g',m',m}^{\Omega} \max(\psi_{p,g',m'}^1(i, j), 0) + \frac{1}{3} K_{g,g'}^{E,2} K_{g',m',m}^{\Omega} \psi_{p,g',m'}^2(i, j) \right]. \tag{7}$$

这是代码对 Eq. (10b) 中 catastrophic secondary source 的离散实现。

## 5. Energy Step: Eq. (15), 使用 CN

energy step 对每个固定的  $(i, j, m)$  沿能量组  $g$  从高能到低能递推。设当前旧值为

$$P_g = \psi_g^{1,n}, \quad R_g = \psi_g^{2,n}, \quad (8)$$

新值为

$$\hat{P}_g = \psi_g^{1,n+1}, \quad \hat{R}_g = \psi_g^{2,n+1}. \quad (9)$$

在代码里

$$S_g = S_s[g], \quad S_{g+1} = S_s[g+1], \quad \sigma_g = \max(\sigma_{c,g}, 0), \quad h = \Delta x_{\mathcal{E}}, \quad \Delta E = \text{dg}. \quad (10)$$

其中  $h = \Delta x/2$ 。

代码使用的 Eq. (15) DG/CN 更新可写成下列代数形式。这里是当前实现中第一处使用 Crank–Nicolson 的地方；每个 Strang 步中调用两次，且每次传入半步  $h = \Delta x/2$ 。先定义

$$D_g = \frac{1}{h} + \frac{1}{2}\rho \frac{S_g}{\Delta E} + \frac{1}{2}\sigma_g, \quad A_g = \frac{3}{2}\rho \frac{S_{g+1}}{\Delta E} \frac{1}{D_g}, \quad (11)$$

$$B_g = \frac{1}{2}\rho \frac{S_g}{\Delta E} \frac{1}{D_g}, \quad C_g = \frac{1}{2}\rho \frac{S_{g+1}}{\Delta E} \frac{1}{D_g}. \quad (12)$$

并记

$$U_g = P_{g+1} - R_{g+1}, \quad V_g = P_g - R_g. \quad (13)$$

若启用 straggling，代码加入

$$\mathcal{S}_g = \frac{1}{2}\rho \frac{T_{g+1}}{\Delta E^2} P_{g+1} + \frac{1}{2}\rho \frac{T_{g-1}}{\Delta E^2} P_{g-1} - \rho \frac{T_g}{\Delta E^2} P_g. \quad (14)$$

若未启用，则  $\mathcal{S}_g = 0$ 。

定义

$$M_g = \frac{1}{2}\rho \frac{S_{g+1}}{\Delta E} U_g - \frac{1}{2}\rho \frac{S_g}{\Delta E} V_g - \frac{1}{2}\sigma_g P_g + \frac{P_g}{h} + \mathcal{S}_g + Q_g^1. \quad (15)$$

则

$$R_g^{(4)} = \frac{M_g}{D_g}. \quad (16)$$

第二个 DG 系数满足

$$\hat{R}_g = \frac{N_g}{C_g^R}, \quad (17)$$

其中

$$\begin{aligned} N_g = & R_g/h - \frac{1}{2}\sigma_g R_g + \frac{3}{2}\rho \frac{S_{g+1}}{\Delta E} U_g + \frac{3}{2}\rho \frac{S_g}{\Delta E} V_g \\ & - \frac{3}{2}\rho \frac{S_g + S_{g+1}}{\Delta E} P_g - \frac{1}{2}\rho \frac{S_{g+1} - S_g}{\Delta E} R_g - A_g M_g + Q_g^2 \\ & + H_g(\hat{P}_{g+1} - \hat{R}_{g+1}), \end{aligned} \quad (18)$$

$$C_g^R = \frac{1}{h} + \frac{1}{2}\sigma_g + \frac{3}{2}\rho \frac{S_g}{\Delta E} + \frac{1}{2}\rho \frac{S_{g+1} - S_g}{\Delta E} + A_g \frac{1}{2}\rho \frac{S_g}{\Delta E}. \quad (19)$$

这里

$$H_g = \frac{3}{2}\rho \frac{S_{g+1}}{\Delta E} - A_g \frac{1}{2}\rho \frac{S_{g+1}}{\Delta E}. \quad (20)$$

最后

$$\hat{P}_g = B_g \hat{R}_g + R_g^{(4)} + C_g(\hat{P}_{g+1} - \hat{R}_{g+1}). \quad (21)$$

主粒子时  $Q_g^1 = Q_g^2 = 0$ 。二次粒子时  $Q_g^1, Q_g^2$  来自上面的 secondary source。

### 5.1 Energy Step 的实际更新顺序

上式不是同时求解  $\hat{P}_g$  和  $\hat{R}_g$ ，而是沿能量组从高能到低能反向扫掠。原因是当前组  $g$  的新值依赖高能邻居的新值差

$$\Delta_{g+1} = \hat{P}_{g+1} - \hat{R}_{g+1}. \quad (22)$$

代码初始化最高能边界为

$$\hat{P}_{N_g} = 0, \quad \hat{R}_{N_g} = 0, \quad (23)$$

对应 `next_F = 0.0` 和 `next_f = 0.0`。

对每个能量组  $g = N_g - 1, N_g - 2, \dots, 0$ ，代码执行：

$$\Delta_{g+1} = \hat{P}_{g+1} - \hat{R}_{g+1}, \quad (24)$$

$$\hat{R}_g = \frac{N_g^0 + H_g \Delta_{g+1}}{C_g^R}, \quad (25)$$

$$\hat{P}_g = B_g \hat{R}_g + R_g^{(4)} + C_g \Delta_{g+1}. \quad (26)$$

其中  $N_g^0$  是不含高能新值差  $\Delta_{g+1}$  的分子部分。

写成代码变量就是

$$\begin{aligned} \hat{R}_g &\leftrightarrow \text{f\_val}, \\ \hat{P}_g &\leftrightarrow \text{F\_val}, \\ \Delta_{g+1} &\leftrightarrow \text{hi\_delta} = \text{next\_F} - \text{next\_f}, \\ B_g &\leftrightarrow \text{F\_from\_f}[\text{idx}], \\ C_g &\leftrightarrow \text{F\_from\_hi}[\text{idx}], \\ R_g^{(4)} &\leftrightarrow \text{rhs4}[\text{idx}], \\ N_g^0 &\leftrightarrow \text{rhs\_base}[\text{idx}], \\ H_g &\leftrightarrow \text{rhs\_high\_coeff}[\text{idx}], \\ C_g^R &\leftrightarrow \text{coe}[\text{idx}]. \end{aligned} \quad (27)$$

对应代码更新为

$$\text{f\_val} = \frac{\text{rhs\_base}[\text{idx}] + \text{rhs\_high\_coeff}[\text{idx}] \cdot \text{hi\_delta}}{\text{coe}[\text{idx}]}, \quad (28)$$

$$\text{F\_val} = \text{F\_from\_f}[\text{idx}] \cdot \text{f\_val} + \text{rhs4}[\text{idx}] + \text{F\_from\_hi}[\text{idx}] \cdot \text{hi\_delta}. \quad (29)$$

随后代码令

$$\text{next\_F} = \text{F\_val}, \quad \text{next\_f} = \text{f\_val}, \quad (30)$$

供下一个低能组  $g - 1$  使用。

### 5.2 Energy Step 中没有显式三角矩阵

5.1 中的反向扫掠可以从矩阵角度理解为一个能量方向的三角递推系统：当前组  $g$  的新值只依赖已经算出的高能邻居差值  $\Delta_{g+1} = \hat{P}_{g+1} - \hat{R}_{g+1}$ 。因此若把所有能量组一次性排成全局线性系统，它会具有三角/递推结构。

但当前 CUDA 实现没有显式组装这个三角矩阵，也没有调用 Thomas 算法或通用线性求解器。代码采用的是 5.1 所写的逐组计算方式：先由旧值和材料系数计算

$$\begin{aligned} & \text{rhs4}[\text{idx}], \quad \text{rhs\_base}[\text{idx}], \quad \text{rhs\_high\_coeff}[\text{idx}], \\ & \text{F\_from\_f}[\text{idx}], \quad \text{F\_from\_hi}[\text{idx}], \quad \text{coe}[\text{idx}]. \end{aligned} \quad (31)$$

然后在同一次 energy step 中按  $g = N_g - 1, N_g - 2, \dots, 0$  直接计算  $\text{f\_val}$  与  $\text{F\_val}$ 。

也就是说，三角矩阵只是对 5.1 递推公式的等价解释；实际代码并不存储矩阵元素。每个能量组只使用当前组的预计算系数和上一轮反向扫掠留下的  $\text{next\_F}$ ,  $\text{next\_f}$ ：

$$\text{hi\_delta} = \text{next\_F} - \text{next\_f}, \quad (32)$$

再现场计算

$$\text{f\_val} = \frac{\text{rhs\_base}[\text{idx}] + \text{rhs\_high\_coeff}[\text{idx}] \cdot \text{hi\_delta}}{\text{coe}[\text{idx}]}, \quad (33)$$

$$\text{F\_val} = \text{F\_from\_f}[\text{idx}] \cdot \text{f\_val} + \text{rhs4}[\text{idx}] + \text{F\_from\_hi}[\text{idx}] \cdot \text{hi\_delta}. \quad (34)$$

随后更新  $\text{next\_F} = \text{F\_val}$ ,  $\text{next\_f} = \text{f\_val}$ ，继续处理下一个低能组。

## 6. Transport Step: Eq. (18), Eq. (22)，当前代码未使用 CN

固定能量组  $g$  和角度方向  $m$ ，transport 子步求解

$$\partial_x \psi + \Omega_{m,y} \partial_y \psi + \Omega_{m,z} \partial_z \psi = 0. \quad (35)$$

这是论文 Eq. (18) 的  $L_1$  子问题；代码中 Eq. (22) 用 MUSCL 有限体积通量实现。当前实现没有求解

$$\psi^{n+1} = \psi^n - \frac{h}{2} [L_1(\psi^n) + L_1(\psi^{n+1})] \quad (36)$$

这样的隐式/CN transport 系统，而是使用下面的显式 predictor-corrector。

MUSCL slope limiter 为

$$\theta_{i,j}^y = \frac{\psi_{i,j} - \psi_{i-1,j}}{\psi_{i+1,j} - \psi_{i,j}}, \quad \eta(\theta) = \max\{0, \max[\min(2\theta, 1), \min(\theta, 2)]\}, \quad (37)$$

$$\sigma_{i,j}^y = \eta(\theta_{i,j}^y)(\psi_{i+1,j} - \psi_{i,j}), \quad \sigma_{i,j}^z = \eta(\theta_{i,j}^z)(\psi_{i,j+1} - \psi_{i,j}). \quad (38)$$

界面重构为

$$\psi_{i+1/2,j}^- = \psi_{i,j} + \frac{1}{2}\sigma_{i,j}^y, \quad \psi_{i+1/2,j}^+ = \psi_{i+1,j} - \frac{1}{2}\sigma_{i+1,j}^y, \quad (39)$$

$$\psi_{i,j+1/2}^- = \psi_{i,j} + \frac{1}{2}\sigma_{i,j}^z, \quad \psi_{i,j+1/2}^+ = \psi_{i,j+1} - \frac{1}{2}\sigma_{i,j+1}^z. \quad (40)$$

上风数值通量为

$$H_{i+1/2,j}^y = \Omega_{m,y} \begin{cases} \psi_{i+1/2,j}^-, & \Omega_{m,y} \geq 0, \\ \psi_{i+1/2,j}^+, & \Omega_{m,y} < 0, \end{cases} \quad (41)$$

$$H_{i,j+1/2}^z = \Omega_{m,z} \begin{cases} \psi_{i,j+1/2}^-, & \Omega_{m,z} \geq 0, \\ \psi_{i,j+1/2}^+, & \Omega_{m,z} < 0. \end{cases} \quad (42)$$

因此

$$L_1(\psi_{i,j,g,m}) = \frac{H_{i+1/2,j}^y - H_{i-1/2,j}^y}{\Delta y} + \frac{H_{i,j+1/2}^z - H_{i,j-1/2}^z}{\Delta z}. \quad (43)$$

代码采用显式 predictor-corrector:

$$\psi_{i,j,g,m}^* = \psi_{i,j,g,m}^n - h L_1(\psi_{i,j,g,m}^n), \quad (44)$$

$$\psi_{i,j,g,m}^{n+1} = \psi_{i,j,g,m}^n - \frac{h}{2} [L_1(\psi_{i,j,g,m}^n) + L_1(\psi_{i,j,g,m}^*)], \quad (45)$$

其中  $h = \Delta x/2$ 。

## 7. Angle Diffusion Step: Eq. (19), Eq. (20), Eq. (21), 使用 CN

固定  $(i, j, g)$ , 角扩散子步求解论文 Eq. (19):

$$\partial_x \psi = D_g(\partial_{uu} \psi + \partial_{vv} \psi), \quad D_g = \frac{1}{2} \rho \Sigma_{\text{tr},g}. \quad (46)$$

代码使用零边界 sine basis, 对应论文 Eq. (20), Eq. (21)。角网格内点为

$$u_a = -\frac{L_u}{2} + (a+1)\Delta u, \quad v_b = -\frac{L_v}{2} + (b+1)\Delta v. \quad (47)$$

sine 模态为

$$s_{\mu,a}^u = \sin \frac{\pi(\mu+1)(a+1)}{N_u+1}, \quad s_{\nu,b}^v = \sin \frac{\pi(\nu+1)(b+1)}{N_v+1}. \quad (48)$$

离散 Laplacian 本征值为

$$\lambda_\mu^u = \frac{2 - 2 \cos \left( \frac{\pi(\mu+1)}{N_u+1} \right)}{\Delta u^2}, \quad \lambda_\nu^v = \frac{2 - 2 \cos \left( \frac{\pi(\nu+1)}{N_v+1} \right)}{\Delta v^2}. \quad (49)$$

这是当前实现中第二处使用 Crank-Nicolson 的地方。代码先用 sine/DST 将角扩散算子对角化, 然后对每个谱模态使用 CN 更新因子

$$r_{g,\mu,\nu} = \frac{2/\Delta x - D_g(\lambda_\mu^u + \lambda_\nu^v)}{2/\Delta x + D_g(\lambda_\mu^u + \lambda_\nu^v)}. \quad (50)$$

因此角扩散完整步为

$$\hat{\psi}_{g,\mu,\nu} = r_{g,\mu,\nu} \psi_{g,\mu,\nu}, \quad (51)$$

再做 inverse sine transform 回到角网格。

### 7.1 统一角网格路径: Separable Real DST

旧的大角度网格路径使用 odd extension 加 2D complex FFT:

$$\text{pack odd extension} \rightarrow \text{2D C2C FFT} \rightarrow \text{multiply } r_{g,\mu,\nu} \rightarrow \text{2D inverse C2C FFT} \rightarrow \text{unpack}. \quad (52)$$

该方法保持了 sine basis 的数学形式, 但会把角网格从

$$N_u \times N_v \quad (53)$$

扩展为

$$2(N_u + 1) \times 2(N_v + 1), \quad (54)$$

并且使用 complex FFT，导致内存流量和 workspace 都较大。

当前代码对所有角网格大小统一使用 separable real DST kernel 路径。也就是说，小网格不再切到 direct DST kernel，大网格也不再切到 odd-extension FFT fallback；所有  $N_u, N_v$  都按同一个 real-valued separable sine transform 流程执行。

由于角扩散算子是 separable 的，

$$\partial_{uu} + \partial_{vv}, \quad (55)$$

二维 sine transform 可以拆成两个一维 sine transform。令

$$\in \mathbb{R}^{N_v \times N_u}, \quad (56)$$

其中行方向为  $v$ ，列方向为  $u$ 。定义 sine 矩阵

$$(S_u)_{\mu,a} = \sin \frac{\pi(\mu+1)(a+1)}{N_u+1}, \quad (S_v)_{\nu,b} = \sin \frac{\pi(\nu+1)(b+1)}{N_v+1}. \quad (57)$$

forward DST 可以写成矩阵形式：

$$\tilde{\sim} = \alpha S_v^T S_u^T, \quad \alpha = \frac{4}{(N_u+1)(N_v+1)}. \quad (58)$$

CN 谱更新为。这里的 new 是新深度步的状态标签，不表示幂次；为避免和幂次混淆，下面统一把它写成下标。

$$\tilde{\sim}_{\text{new}, \nu, \mu} = r_{g, \mu, \nu} \tilde{\sim}_{\nu, \mu}. \quad (59)$$

inverse DST 为

$$\text{new} = S_v^T \tilde{\sim}_{\text{new}} S_u. \quad (60)$$

合并起来就是

$$\boxed{\text{new} = S_v^T [\mathbf{R}_g \odot (\alpha S_v S_u^T)] S_u}, \quad (61)$$

其中  $\mathbf{R}_g = (r_{g, \mu, \nu})$ ， $\odot$  表示逐元素乘。

实现上可以用 batched GEMM 或 fused shared-memory kernel：

$$\begin{aligned} \mathbf{Y} &= S_u^T, \\ \mathbf{Z} &= S_v \mathbf{Y}, \\ \mathbf{Z}_{\nu, \mu} &\leftarrow \alpha r_{g, \mu, \nu} \mathbf{Z}_{\nu, \mu}, \\ \mathbf{W} &= S_v^T \mathbf{Z}, \\ \text{new} &= \mathbf{W} S_u. \end{aligned} \quad (62)$$

这里每个 batch 对应一个固定的  $(y_i, z_j, E_g)$  平面。代码中的 batch 数为

$$N_{\text{batch}} = (N_y + 1)(N_z + 1)N_g. \quad (63)$$

该优化的优点是：

- 不改变 Eq. (19)–(21) 的数学离散，只改变 sine transform 的实现方式。

- 避免 odd extension, 不再把  $N_u \times N_v$  扩展到  $2(N_u + 1) \times 2(N_v + 1)$ 。
- 避免 complex FFT, 数据保持 real-valued。
- 对小网格和大网格使用同一条 kernel 路径, 减少分支间数值验证和维护成本。
- CN factor multiplication 可以和中间矩阵  $\mathbf{Z}$  的写回融合, 减少一次全局内存读写。

当前实现策略为:

$$\begin{aligned} & \text{all angular grids} \\ & \rightarrow \text{separable real DST kernels.} \end{aligned} \quad (64)$$

当前代码实现采用的分支为

$$\text{all } N_u, N_v : \text{separable real DST 路径.} \quad (65)$$

当前 separable 路径先用四个 real-valued CUDA kernel 实现上述矩阵分解; 后续还可以进一步替换为 cuBLAS batched GEMM 或 shared-memory tiled kernel。

## 7.2 Separable DST 当前实现的详细步骤

对每个固定的  $(y_i, z_j, E_g)$ , 角平面是一个实矩阵

$$\in \mathbb{R}^{N_v \times N_u}. \quad (66)$$

其中  $N_u = \text{Nom}$ ,  $N_v = \text{Nmu}$ 。代码实际存储是 angle-major 的一维数组:

$$\text{angle} = bN_u + a, \quad a = 0, \dots, N_u - 1, \quad b = 0, \dots, N_v - 1. \quad (67)$$

batch 编号对应

$$\text{batch index} = g(N_y + 1)(N_z + 1) + \text{yz cell}. \quad (68)$$

当前 separable real DST 路径按四个阶段执行。

$Y, Z, W$  与抽象  $S, \Lambda$  的对应关系 前面写的

$$L = S^{-1} \Lambda S \quad (69)$$

是抽象的一维向量写法: 把整个二维角平面  $\Psi$  拉直成向量  $\psi$ , 然后用一个完整二维 DST 矩阵  $S$  做变换。在这个抽象写法中, angle step 等价于

$$\psi_{\text{new}} = S^{-1} R_g S \psi, \quad (70)$$

其中

$$R_g = \left( I - \frac{\Delta x}{2} D_g \Lambda \right)^{-1} \left( I + \frac{\Delta x}{2} D_g \Lambda \right). \quad (71)$$

因为  $\Lambda$  是对角矩阵,  $R_g$  也是对角矩阵; 其对角元就是代码中的

$$r_{g,\mu,\nu}. \quad (72)$$



代码没有显式构造完整二维矩阵  $S$ 。它利用二维 DST 的可分离性，把

$$S\psi \quad (73)$$

写成两个方向的矩阵乘：

$$S\psi \iff \alpha S_v \Psi S_u^T. \quad (74)$$

其中  $S_u$  和  $S_v$  分别是一维 sine 矩阵， $\alpha$  是 DST 归一化因子。同理，完整二维 inverse DST 不是单独的  $S_v^T$ ，而是

$$S^{-1}(\cdot) \iff S_v^T(\cdot)S_u. \quad (75)$$

也就是说， $S_v^T$  只对应  $v$  方向的 inverse sine transform；右乘  $S_u$  对应  $u$  方向的 inverse sine transform。两步合起来才是完整的  $S^{-1}$ 。

于是中间矩阵  $Y, Z, W$  的含义是：

$$\begin{aligned} \mathbf{Y} &= \Psi S_u^T & : & \text{完成 } u \text{ 方向 forward DST, 是 } S\psi \text{ 的第一半,} \\ \mathbf{Z} &= \mathbf{R}_g \odot (\alpha S_v \mathbf{Y}) & : & \text{完成 } v \text{ 方向 forward DST, 并乘 CN 因子,} \\ \mathbf{W} &= S_v^T \mathbf{Z} & : & \text{完成 } v \text{ 方向 inverse DST, 是 } S^{-1} \text{ 的第一半,} \\ \Psi_{\text{new}} &= \mathbf{W} S_u & : & \text{完成 } u \text{ 方向 inverse DST, 得到新角分布.} \end{aligned} \quad (76)$$

因此

$$\boxed{\Psi_{\text{new}} = S_v^T [\mathbf{R}_g \odot (\alpha S_v \Psi S_u^T)] S_u} \quad (77)$$

就是矩阵形式的

$$\boxed{\psi_{\text{new}} = S^{-1} R_g S \psi.} \quad (78)$$

对应关系可以简写为

$$\begin{aligned} S &\iff \alpha S_v(\cdot) S_u^T, \\ R_g &\iff \mathbf{R}_g \odot (\cdot), \\ S^{-1} &\iff S_v^T(\cdot) S_u. \end{aligned} \quad (79)$$

这里  $\Lambda$  没有显式出现，是因为它已经被吸收到 CN 更新因子

$$r_{g,\mu,\nu} = \frac{2/\Delta x - D_g \lambda_{\mu,\nu}^+}{2/\Delta x + D_g \lambda_{\mu,\nu}^+} \quad (80)$$

中了。

$S_u$  和  $S_u^T$  的含义 定义

$$(S_u)_{\mu,a} = \sin \frac{\pi(\mu+1)(a+1)}{N_u+1}, \quad \mu = 0, \dots, N_u-1, \quad a = 0, \dots, N_u-1. \quad (81)$$

这里  $a$  是原始  $u$  网格点编号， $\mu$  是  $u$  方向 sine mode 编号。因此

$$S_u \in \mathbb{R}^{N_u \times N_u}, \quad (82)$$

其行对应 mode  $\mu$ ，列对应 grid index  $a$ 。于是

$$(S_u^T)_{a,\mu} = (S_u)_{\mu,a}. \quad (83)$$

因为角平面矩阵写成

$$\in \mathbb{R}^{N_v \times N_u}, \quad (84)$$

其中列方向是  $u$ ，所以右乘  $S_u^T$ ：

$$\mathbf{Y} = S_u^T \quad (85)$$

就是对每一行  $v_b$  沿  $u$  方向做 forward sine transform。逐元素写为

$$Y_{b,\mu} = \sum_{a=0}^{N_u-1} \Psi_{b,a} (S_u^T)_{a,\mu} = \sum_{a=0}^{N_u-1} \Psi_{b,a} \sin \frac{\pi(\mu+1)(a+1)}{N_u+1}. \quad (86)$$

维度为

$$(N_v \times N_u)(N_u \times N_u) = N_v \times N_u. \quad (87)$$

同理，左乘  $S_v$  是沿  $v$  方向做 forward sine transform，左乘  $S_v^T$  是沿  $v$  方向做 inverse sine transform，右乘  $S_u$  是沿  $u$  方向做 inverse sine transform。

**CN 矩阵方程的来源** angle diffusion 子步的连续方程是

$$\partial_x \psi = D_g (\partial_{uu} \psi + \partial_{vv} \psi). \quad (88)$$

把角变量  $(u, v)$  离散后，所有角网格点上的值排成向量

$$\boldsymbol{\psi} = (\psi_{0,0}, \psi_{1,0}, \dots, \psi_{N_u-1, N_v-1})^T. \quad (89)$$

离散角 Laplacian 记为矩阵  $L$ ，即

$$L\boldsymbol{\psi} \approx \partial_{uu}\psi + \partial_{vv}\psi. \quad (90)$$

$L\boldsymbol{\psi}$  和原始的  $\boldsymbol{\psi}$  的关系是： $\boldsymbol{\psi}$  是当前角网格上的函数值，而  $L\boldsymbol{\psi}$  是对这些函数值做二阶差分后得到的角向曲率/扩散项。它不是另一个独立未知量，而是由当前  $\boldsymbol{\psi}$  线性计算出来的结果。逐点写为

$$(L\boldsymbol{\psi})_{a,b} = \frac{\psi_{a+1,b} - 2\psi_{a,b} + \psi_{a-1,b}}{\Delta u^2} + \frac{\psi_{a,b+1} - 2\psi_{a,b} + \psi_{a,b-1}}{\Delta v^2}. \quad (91)$$

这里边界采用零边界条件；因此边界外侧的值按零处理。直观上，如果  $\psi$  在角空间很平滑， $L\boldsymbol{\psi}$  较小；如果  $\psi$  在角空间弯曲/变化很快， $L\boldsymbol{\psi}$  较大。angle diffusion step 正是用  $D_g L\boldsymbol{\psi}$  来描述角分布在  $u, v$  方向的扩散。于是得到关于深度变量  $x$  的 ODE 系统

$$\frac{d\boldsymbol{\psi}}{dx} = D_g L\boldsymbol{\psi}. \quad (92)$$

对这个 ODE 系统从  $x^n$  到  $x^{n+1} = x^n + \Delta x$  使用 Crank-Nicolson：

$$\frac{\boldsymbol{\psi}^{n+1} - \boldsymbol{\psi}^n}{\Delta x} = \frac{1}{2} D_g L\boldsymbol{\psi}^n + \frac{1}{2} D_g L\boldsymbol{\psi}^{n+1}. \quad (93)$$

两边乘以  $\Delta x$  并把含  $\boldsymbol{\psi}^{n+1}$  的项移到左边：

$$\boldsymbol{\psi}^{n+1} - \frac{\Delta x}{2} D_g L\boldsymbol{\psi}^{n+1} = \boldsymbol{\psi}^n + \frac{\Delta x}{2} D_g L\boldsymbol{\psi}^n. \quad (94)$$

因此得到矩阵方程

$$\left( I - \frac{\Delta x}{2} D_g L \right) \boldsymbol{\psi}^{n+1} = \left( I + \frac{\Delta x}{2} D_g L \right) \boldsymbol{\psi}^n. \quad (95)$$

这就是 angle step 中 CN 矩阵方程的来源。DST 的作用是把  $L$  对角化，从而避免直接求解这个大线性系统。

如何利用  $L = S^{-1}\Lambda S$  对角化 CN 系统 从 CN 矩阵方程出发：

$$\left(I - \frac{\Delta x}{2}D_g L\right)\psi^{n+1} = \left(I + \frac{\Delta x}{2}D_g L\right)\psi^n. \quad (96)$$

设离散角 Laplacian 可以被 sine transform 对角化：

$$L = S^{-1}\Lambda S, \quad (97)$$

其中  $S$  表示二维 DST 变换,  $\Lambda$  是对角矩阵：

$$\Lambda = \text{diag}(\lambda_{\mu,\nu}). \quad (98)$$

定义 mode 空间变量

$$\tilde{\psi} = S\psi, \quad \psi = S^{-1}\tilde{\psi}. \quad (99)$$

把

$$\psi^{n+1} = S^{-1}\tilde{\psi}^{n+1}, \quad \psi^n = S^{-1}\tilde{\psi}^n \quad (100)$$

代入 CN 方程：

$$\left(I - \frac{\Delta x}{2}D_g S^{-1}\Lambda S\right)S^{-1}\tilde{\psi}^{n+1} = \left(I + \frac{\Delta x}{2}D_g S^{-1}\Lambda S\right)S^{-1}\tilde{\psi}^n. \quad (101)$$

利用  $SS^{-1} = I$ , 并在方程两边左乘  $S$ , 得到

$$\left(I - \frac{\Delta x}{2}D_g \Lambda\right)\tilde{\psi}^{n+1} = \left(I + \frac{\Delta x}{2}D_g \Lambda\right)\tilde{\psi}^n. \quad (102)$$

这里需要注意：不是单独把  $D_g$  右侧的  $S^{-1}$  去掉，而是整个方程两边左乘  $S$  后成对相消。令

$$a = \frac{\Delta x}{2}. \quad (103)$$

以左边为例，先展开

$$(I - aD_g S^{-1}\Lambda S)S^{-1}\tilde{\psi}^{n+1} = S^{-1}\tilde{\psi}^{n+1} - aD_g S^{-1}\Lambda SS^{-1}\tilde{\psi}^{n+1}. \quad (104)$$

由于  $SS^{-1} = I$ , 上式变成

$$S^{-1}\tilde{\psi}^{n+1} - aD_g S^{-1}\Lambda \tilde{\psi}^{n+1}. \quad (105)$$

这时最前面的  $S^{-1}$  还在。接着对整个方程左乘  $S$ , 得到

$$S\left[S^{-1}\tilde{\psi}^{n+1} - aD_g S^{-1}\Lambda \tilde{\psi}^{n+1}\right] = \tilde{\psi}^{n+1} - aD_g \Lambda \tilde{\psi}^{n+1}. \quad (106)$$

这里  $D_g$  是标量，因此可与矩阵乘法交换；真正消掉的是

$$SS^{-1} = I. \quad (107)$$

右边同理，因此最后得到 mode 空间中的对角 CN 系统。由于  $\Lambda$  是对角矩阵，这个系统已经完全解耦。

对每个 mode  $(\mu, \nu)$ , 有

$$\left(1 - \frac{\Delta x}{2}D_g \lambda_{\mu,\nu}\right)\tilde{\psi}_{\mu,\nu}^{n+1} = \left(1 + \frac{\Delta x}{2}D_g \lambda_{\mu,\nu}\right)\tilde{\psi}_{\mu,\nu}^n. \quad (108)$$

因此

$$\tilde{\psi}_{\mu,\nu}^{n+1} = \frac{1 + \frac{\Delta x}{2} D_g \lambda_{\mu,\nu}}{1 - \frac{\Delta x}{2} D_g \lambda_{\mu,\nu}} \tilde{\psi}_{\mu,\nu}^n. \quad (109)$$

代码中使用的离散 Laplacian 特征值按正数

$$\lambda_{\mu,\nu}^+ = \lambda_{\mu}^u + \lambda_{\nu}^v \quad (110)$$

存储，而扩散算子实际对应  $-\lambda_{\mu,\nu}^+$ 。因此代码里的 CN 因子写成

$$r_{g,\mu,\nu} = \frac{2/\Delta x - D_g \lambda_{\mu,\nu}^+}{2/\Delta x + D_g \lambda_{\mu,\nu}^+}. \quad (111)$$

这正是

$$\text{buildDiffusionFactorKernel} \quad (112)$$

中构造的谱空间更新因子。

**第一步：沿  $u$  方向 forward DST** 对每个固定的  $v_b$ ，计算

$$Y_{b,\mu} = \sum_{a=0}^{N_u-1} \Psi_{b,a} S_{\mu,a}^u, \quad \mu = 0, \dots, N_u - 1. \quad (113)$$

矩阵形式：

$$\mathbf{Y} = S_u^T, \quad \mathbf{Y} \in \mathbb{R}^{N_v \times N_u}. \quad (114)$$

代码对应：

$$\text{separableForwardUKernel}. \quad (115)$$

**第二步：沿  $v$  方向 forward DST，并乘 CN 因子** 对每个固定的  $u$ -mode  $\mu$ ，计算

$$Z_{\nu,\mu} = \alpha r_{g,\mu,\nu} \sum_{b=0}^{N_v-1} S_{\nu,b}^v Y_{b,\mu}, \quad \nu = 0, \dots, N_v - 1. \quad (116)$$

其中

$$\alpha = \frac{4}{(N_u + 1)(N_v + 1)} \quad (117)$$

是二维 DST-I 的归一化因子，

$$r_{g,\mu,\nu} = \frac{2/\Delta x - D_g(\lambda_{\mu}^u + \lambda_{\nu}^v)}{2/\Delta x + D_g(\lambda_{\mu}^u + \lambda_{\nu}^v)} \quad (118)$$

是 Eq. (19)–(21) 的 CN 谱更新因子。矩阵形式：

$$\mathbf{Z} = \mathbf{R}_g \odot (\alpha S_v \mathbf{Y}). \quad (119)$$

代码对应：

$$\text{separableForwardVFactorKernel}. \quad (120)$$

这里  $\alpha r_{g,\mu,\nu}$  预先存入 `d_sine_coeff`。

**第三步：沿  $v$  方向 inverse DST** 对每个固定的  $\mu$ ，计算

$$W_{b,\mu} = \sum_{\nu=0}^{N_v-1} S_{\nu,b}^v Z_{\nu,\mu}. \quad (121)$$

矩阵形式：

$$\mathbf{W} = S_v^T \mathbf{Z}, \quad \mathbf{W} \in \mathbb{R}^{N_v \times N_u}. \quad (122)$$

代码对应：

$$\text{separableInverseVKernel}. \quad (123)$$

**第四步：沿  $u$  方向 inverse DST** 对每个固定的  $v_b$ ，计算

$$\Psi_{\text{new},b,a} = \sum_{\mu=0}^{N_u-1} W_{b,\mu} S_{\mu,a}^u. \quad (124)$$

矩阵形式：

$$\Psi_{\text{new}} = \mathbf{W} S_u. \quad (125)$$

代码对应：

$$\text{separableInverseUKernel}. \quad (126)$$

综上，当前 separable 路径等价于

$$\Psi_{\text{new}} = S_v^T [\mathbf{R}_g \odot (\alpha S_v S_u^T)] S_u. \quad (127)$$

它和原来的 sine/CN 离散数学上是一致的；区别只是没有通过 odd extension 和 complex FFT 实现。

## 8. 当前 CUDA 并行流程与 streaming 优化

本节按当前代码实现说明并行映射。令

$$N_{yz} = (N_y + 1)(N_z + 1), \quad N_\Omega = N_\mu N_\omega, \quad N_{\text{lane}} = N_{yz} N_\Omega. \quad (128)$$

代码中的主要一维存储布局为

$$\text{idx} = (g N_{yz} + s) N_\Omega + m, \quad (129)$$

其中  $s = j(N_y + 1) + i$ ， $m$  是角度编号。

### 8.1 Energy step 的并行流程

energy step 的关键 kernel 是 `streamingEnergyDgCnKernel`。streaming 模式中先按 lane 切块，`lane_chunk` 会按  $N_\Omega$  对齐，使一个 chunk 不切断角平面。每个 CUDA block 使用 256 个 thread：

$$\text{lane} = \text{blockIdx.x} \cdot 256 + \text{threadIdx.x}. \quad (130)$$

一个 thread 负责一个固定的  $(y, z, \Omega)$  lane，然后在该 lane 内从高能到低能串行扫描

$$g = N_g - 1, N_g - 2, \dots, 0. \quad (131)$$

这样做的原因是 Eq. (15) 的新值依赖高能邻居新值差  $\hat{P}_{g+1} - \hat{R}_{g+1}$ 。这个依赖沿能量方向是递推关系，不能直接把同一个 lane 的所有能量组完全并行化；当前实现把并行度放在  $N_{yz}N_{\Omega}$  个 lane 上。

每个 thread 在寄存器中保留

$$\text{next\_F}, \quad \text{next\_f}, \quad (132)$$

分别表示已算出的高能邻居  $\hat{P}_{g+1}, \hat{R}_{g+1}$ 。对当前能量组  $g$ , thread 从 chunk 内连续内存读取  $F_g, f_g, F_{g+1}, f_{g+1}$  以及需要的材料系数，计算 Eq. (15) 的局部系数，写出  $\hat{P}_g, \hat{R}_g$ ，再更新 **next\_F**, **next\_f**。

二次粒子 energy half-step 需要先由当前主粒子分布构造 catastrophic secondary source。对一个固定输出  $(g, s, m)$ ，代码实现的离散形式可概括为

$$Q_{g,s,m} = \Delta E \sum_{g'=g_{\min}(g)}^{g_{\max}(g)-1} \sum_{m' \in \mathcal{I}(g',m)} \left[ K_{g,g'}^{E,1} \max(\psi_{p,g',s,m'}^1, 0) + \frac{1}{3} K_{g,g'}^{E,2} \psi_{p,g',s,m'}^2 \right] K_{g',m',m}^{\Omega}. \quad (133)$$

这里  $s$  是同一个  $y, z$  cell；source 不在横向空间上耦合，只在能量和角度上从主粒子分布向二次粒子分布积分。

streaming 模式下，二次 source 的数据流为：

$$\begin{aligned} & \text{read primary } (\psi_p^1, \psi_p^2) \text{ lane chunk} \\ & \quad \downarrow \\ & \text{copy to GPU temporary buffers} \\ & \quad \downarrow \\ & \text{streamingSecondarySourceTiledEnergyKernel<4>} \\ & \quad \downarrow \\ & \text{source buffer passed directly to streamingEnergyDgCnKernel.} \end{aligned} \quad (134)$$

也就是说，source 不先写成一个完整的全局数组再读回来；它在当前 lane chunk 的 GPU workspace 中生成，随后立即作为二次粒子 Eq. (15) 右端项使用。

**streamingSecondarySourceTiledEnergyKernel<4>** 的 grid 为

$$\text{grid.x} = \left\lceil \frac{\text{lanes}}{256} \right\rceil, \quad \text{grid.y} = \left\lceil \frac{N_g^{\text{active}}}{4} \right\rceil. \quad (135)$$

其中

$$\text{lane} = \text{blockIdx.x} \cdot 256 + \text{threadIdx.x}, \quad g_0 = 4 \cdot \text{blockIdx.y}. \quad (136)$$

一个 thread 对应一个固定 lane，即固定的  $(s, m)$ ，但一次最多累计 4 个相邻输出能量组  $g_0, g_0+1, g_0+2, g_0+3$ 。这就是模板参数 **OUT\_TILE** = 4 的含义。thread 在寄存器中保留

$$q_1[4], \quad q_2[4], \quad (137)$$

分别累计  $\psi^1$  和  $\psi^2$  对 source 的贡献。

能量方向的优化来自 **ker\_e\_begin** 和 **ker\_e\_end**。对每个输出能量  $g$ ，并不是所有入射能量  $g'$  都有非零贡献；代码预先为每个  $g$  保存有效范围

$$g' \in [\text{ker\_e\_begin}[g], \text{ker\_e\_end}[g]). \quad (138)$$

kernel 对 4 个输出能量组取这些范围的并集，只遍历可能非零的  $g'$ 。进入某个  $g'$  后，thread 先读出该  $g'$  对 4 个输出能量组的  $K^{E,1}, K^{E,2}$ ；如果 4 组全为零，就直接跳过该  $g'$ 。

角度方向的优化来自 sparse column 存储。对固定入射能量  $g'$  和输出角  $m$ ，代码不用遍历所有  $m' = 0, \dots, N_\Omega - 1$ ，而是通过

$$\text{ker\_v\_col\_ptr}[g', m] \rightarrow \text{ker\_v\_col\_ptr}[g', m+1] \quad (139)$$

得到非零角权重列表。每个非零项给出一个输入角偏移  $\text{ker\_v\_in\_idx}[p]$  和权重  $\text{ker\_v\_sparse\_values}[p]$ 。由于 lane chunk 按  $N_\Omega$  对齐，thread 可以用

$$\text{lane\_base} = \text{lane} - m \quad (140)$$

定位同一个  $y, z$  cell 下的各个输入角，读取  $\psi_{p,g',s,m'}^1$  和  $\psi_{p,g',s,m'}^2$ 。角向求和先形成

$$A_1(g') = \sum_{m'} \max(\psi_{p,g',s,m'}^1, 0) K_{g',m',m}^\Omega, \quad A_2(g') = \sum_{m'} \psi_{p,g',s,m'}^2 K_{g',m',m}^\Omega. \quad (141)$$

然后同一个  $A_1, A_2$  同时贡献给 4 个输出能量组：

$$q_1[t] += A_1(g') K_{g_0+t,g'}^{E,1}, \quad q_2[t] += A_2(g') K_{g_0+t,g'}^{E,2}. \quad (142)$$

最后写出

$$\text{source\_F}[(g\_0+t)*\text{lanes} + \text{lane}] = \Delta E \left( q_1[t] + \frac{1}{3} q_2[t] \right). \quad (143)$$

这样做的主要收益是三点：第一，source 的输出能量按 4 组 tile 复用同一次角向求和；第二，能量核和角核都跳过零项，避免稠密  $N_g N_\Omega$  遍历；第三，source buffer 只在当前 streaming chunk 内存在，直接喂给二次粒子的 energy recurrence，减少一次完整 backing-store source 文件读写。

## 8.2 Transport step 的并行流程

transport step 使用显式 MUSCL predictor-corrector。对每个输入数组执行四个 kernel：

$$\begin{aligned} & \text{spatialFluxDivergencePlaneKernel}(F, \text{flux\_old}) \\ & \quad \downarrow \\ & \text{spatialPredictorKernel}(F, \text{flux\_old}, F\_pred) \\ & \quad \downarrow \\ & \text{spatialFluxDivergencePlaneKernel}(F\_pred, \text{flux\_pred}) \\ & \quad \downarrow \\ & \text{spatialCorrectorKernel}(F, \text{flux\_old}, \text{flux\_pred}, F). \end{aligned} \quad (144)$$

其中  $\psi^1$  和  $\psi^2$  两个 DG 系数各调用一次 transport solver； $\psi^1$  默认做空间非负截断， $\psi^2$  保留符号。

$\text{spatialFluxDivergencePlaneKernel}$  的 block 组织不是简单对全数组一维铺开，而是按  $(g, \Omega)$  plane 和  $yz$  tile 组织：

$$\text{plane} = \left\lfloor \frac{\text{blockIdx.x}}{\lceil N_{yz}/256 \rceil} \right\rfloor, \quad s = \text{yz\_block} \cdot 256 + \text{threadIdx.x}. \quad (145)$$

因此一个 thread 负责一个固定的  $(g, \Omega, y, z)$  网格点，读取  $y, z$  方向相邻点，计算 limiter、左右界面上风通量和通量散度。predictor/corrector kernel 则是一维全数组映射：一个 thread 更新一个  $\text{idx}$ 。

### 8.3 Angle step 的并行流程

angle step 对每个固定的  $(g, y, z)$  角平面做 sine/DST 对角化和 CN mode 更新。当前代码不再按角网格大小选择 direct/FFT fallback, 而是对所有大小统一走 separable real DST kernel 路径:

$$\text{all } N_u, N_v : \text{ separable real DST kernel.} \quad (146)$$

本文档中的  $N_\mu = N_\omega = 7$  小网格和图中的  $9 \times 9$  case 都使用 separable real DST。

separable 路径中, 每个 kernel 仍用 256-thread block 一维铺开  $\text{batch} \times N_\Omega$  个任务, 其中  $\text{batch} = N_g N_{yz}$ 。四个 kernel 分别完成

$$\mathbf{Y} = S_u^T, \quad \mathbf{Z} = \mathbf{R}_g \odot (\alpha S_v \mathbf{Y}), \quad \mathbf{W} = S_v^T \mathbf{Z}, \quad \mathbf{new} = \mathbf{W} S_u. \quad (147)$$

每个 thread 计算一个输出元素, 沿一个维度做短循环。相对 odd-extension FFT, 这条路径避免了  $2(N_u + 1) \times 2(N_v + 1)$  的扩展网格和 complex workspace, 数据保持 real-valued。

### 8.4 Streaming 数据读取、缓存与流水

`--streaming-full` 不把完整四个相空间数组常驻 GPU, 而是把  $\psi_p^1, \psi_p^2, \psi_s^1, \psi_s^2$  存在 host backing store 文件中。backing store 是 `--stream-dir` 目录下的临时二进制文件, 逻辑上保存完整相空间数组:

$$\begin{aligned} \text{stream\_F.bin} & : \psi_p^1, \\ \text{stream\_f\_F.bin} & : \psi_p^2, \\ \text{stream\_F1.bin} & : \psi_s^1, \\ \text{stream\_f\_F1.bin} & : \psi_s^2. \end{aligned} \quad (148)$$

这些文件用 `pread/pwrite` 按元素偏移随机读写。代码用 `getStoreFd` 缓存文件描述符 `fd`, 避免每个 chunk 反复 `open/close`, 但这只是文件句柄缓存, 不是数据缓存。

每次处理一个 chunk 的数据路径为:

$$\text{backing store} \rightarrow \text{pinned host buffer} \rightarrow \text{GPU workspace} \rightarrow \text{pinned host buffer} \rightarrow \text{backing store.} \quad (149)$$

pinned host buffer 由 `HostDoubleBuffer` 管理。它优先调用 `cudaMallocHost` 分配 page-locked host memory; 如果分配失败, 才退回普通 pageable `std::vector<double>`。pinned memory 的作用是作为 CPU 和 GPU 之间的高速 staging buffer, 让 `cudaMemcpyAsync` 可以更高效地做 host-to-device 和 device-to-host 拷贝。

需要特别说明: 当前 pinned host buffer 不是 LRU cache。它不记录最近访问的 chunk, 也不保存多个历史 chunk 以供命中复用。它只是固定数量的临时装载区:

$$\begin{aligned} \text{h\_stream\_lane, h\_stream\_chunk} & : \psi^1 \text{ 的两个 staging slot,} \\ \text{h\_stream\_f\_lane, h\_stream\_f\_chunk} & : \psi^2 \text{ 的两个 staging slot,} \\ \text{h\_stream\_primary\_lane, h\_stream\_primary\_f\_lane} & : \text{二次 source 读取主粒子时的 staging buffer.} \end{aligned} \quad (150)$$

当处理下一个 chunk 时, 这些 buffer 会被新数据覆盖; 旧 chunk 是否还在 backing store 中, 只由文件保存, 不由 host buffer 缓存。



双缓冲的流水方式如下。第  $k$  个 chunk 使用  $\text{slot} = k \bmod 2$ 。如果这个 slot 上一次还有异步写回未完成，代码先等待 `pending_write[slot].get()`。随后读入当前 chunk 的  $F$  和  $f$ ：

```
std::async(read f)
read F on current thread
wait for f read
```

(151)

读完后用 `cudaMemcpyAsync` 把两组 host buffer 拷到 GPU workspace。GPU kernel 完成后，结果再异步拷回同一个 slot 的 host buffer。最后，写回 backing store 时再次并行写  $F$  和  $f$ ：

```
pending_write[slot] = std::async(...)
std::async(write f)
write F on write task
wait for f write.
```

(152)

因此它优化的是 I/O 与 GPU 计算之间的重叠，以及  $F/f$  两个文件之间的并行读写，不是通过 LRU 命中减少读取。

energy step 和 transport/angle step 的 chunk 方向不同。energy step 按 lane chunk 读取：

$$\text{chunk shape} \approx N_g^{\text{active}} \times N_{\text{lane, chunk}}. \quad (153)$$

这样每个 GPU thread 可以拿到一个 lane 上连续的能量列，并从高能到低能做 Eq. (15) 递推。transport/angle step 按 energy chunk 读取：

$$\text{chunk shape} \approx N_{g, \text{chunk}} \times N_{yz} \times N_{\Omega}. \quad (154)$$

这样一个 chunk 内包含若干完整能量组的  $(y, z, \Omega)$  平面，适合 transport 的  $y, z$  邻点访问和 angle 的每个  $(g, y, z)$  角平面 DST。

当前真正具有“状态复用”含义的是两类轻量元数据，而不是 pinned data cache。第一类是 `stream_max_active_energy`，它记录每个 backing store 当前还可能非零的最高能量组。如果高能组已经衰减为空，后续 streaming energy step 只读取

$$N_g^{\text{active}} = \max(1, \text{stream\_max\_active\_energy} + 1) \quad (155)$$

个低能/活跃能量组。超过这个范围的 chunk 直接跳过，profile 中记为 zero chunk skip。第二类是已打开文件描述符 `stream_fds`，它复用文件句柄而不是复用数组数据。

二次 source 还有一个可选的 source backing file 设计，但当前运行中 `secondary source cache off`。也就是说代码选择在需要二次 source 时从主粒子 backing store 读取当前 chunk，直接在 GPU 上重算 source，并把 source buffer 立即传给二次粒子的 energy recurrence。这样通常比把完整 source 写成 backing store 文件、下一半步再读回来更省 I/O。

IDD 计算与最后一次 energy 写回融合：同一个 chunk 中先执行 `pairDepthDoseLaneKernel` 和 GPU reduction，再把更新后的 chunk 拷回 host。这样避免为了 IDD 单独再扫一遍完整 backing store。

**后续数据复用优化方向** 因此，当前 streaming 优化可以概括为：

$$\text{out-of-core staging} + \text{I/O/compute overlap} + \text{zero chunk skip}, \quad (156)$$

而不是 cache-based data reuse。也就是说，pinned host buffer 和双缓冲主要减少等待时间，并没有让同一个 chunk 在多个子步或多个 depth step 之间被反复命中复用。

后续若要继续降低 backing-store 流量，优先方向不是简单增加一个 host LRU cache，而是重排 streaming schedule，让同一份数据在 GPU 上停留更久。可考虑的方案包括：

- **GPU residency 延长：**一个 chunk 读入 GPU 后，尽量在写回前完成更多连续子步，例如把局部可行的 transport-angle-transport-energy 边界处理融合到同一次 residency 中。
- **统一 chunk 方向：**当前 energy 按 lane chunk，transport/angle 按 energy chunk。两种 chunk 方向之间切换会导致反复回到 backing store。若能设计统一或分层 chunk layout，可减少跨子步重读。
- **GPU resident active cache：**对仍活跃、即将再次访问的 chunk 保留在 GPU workspace 或一个受限 GPU cache 中，而不是每个子步都写回文件再读出。这里需要显式处理容量限制和写回一致性。
- **host LRU cache 作为次选：**host 侧 LRU 可以减少磁盘读写，但仍要经过 H2D/D2H；只有当 backing store I/O 明显成为瓶颈且 GPU residency 难以继续延长时才值得实现。
- **source 与 IDD 继续融合：**当前 source 已经避免完整 source backing file，IDD 也与 energy 写回融合；后续可继续检查其它 diagnostics 是否还能合并到已有 chunk pass 中。

优先级上，最有价值的是减少 chunk 被写回后又立即读出的次数，也就是优化数据在 GPU 上的 resident time，而不是单纯扩大 pinned host buffer。

## 9. 小网格计时、IDD 与 Table 2 对比

本节使用回退到“230 MeV 偏高”之前的代码路径后重新生成的两组小网格数据：8x8-7x7-Nx1000 与 12x12-9x9-Nx2000。两组均使用 Eq.15 energy model、streaming-full out-of-core、energy-chunk=64、lane-chunk=262144 和 idd-stride=1。当前二进制对这两个角网格均走 separable real DST kernel 路径。

$$(N_x, N_y, N_z, N_g, N_\mu, N_\omega) = (1000, 8, 8, 500, 7, 7) \quad \text{and} \quad (2000, 12, 12, 500, 9, 9), \quad x_{\max} = 40 \text{ cm}.$$

## 9.1 时间总结

| grid             | case      | total s | s/step  | energy s | transport s | angle s |
|------------------|-----------|---------|---------|----------|-------------|---------|
| 8x8-7x7-Nx1000   | water 100 | 99.253  | 0.09925 | 60.910   | 6.575       | 1.198   |
| 8x8-7x7-Nx1000   | water 230 | 174.916 | 0.17492 | 111.166  | 10.754      | 1.823   |
| 8x8-7x7-Nx1000   | bone 100  | 98.985  | 0.09898 | 60.787   | 6.630       | 1.193   |
| 8x8-7x7-Nx1000   | bone 230  | 177.303 | 0.17730 | 112.056  | 11.279      | 1.896   |
| 12x12-9x9-Nx2000 | water 100 | 564.906 | 0.28245 | 344.775  | 35.628      | 4.468   |
| 12x12-9x9-Nx2000 | water 230 | 998.575 | 0.49929 | 632.103  | 55.619      | 7.400   |
| 12x12-9x9-Nx2000 | bone 100  | 558.692 | 0.27935 | 338.871  | 36.161      | 4.466   |
| 12x12-9x9-Nx2000 | bone 230  | 993.284 | 0.49664 | 624.786  | 58.465      | 7.372   |

上表中的 `energy/transport/angle` 是 CUDA event 记录的主要计算段时间; `total` 是每个 time step 的 wall-clock 汇总, 包含 host backing-store 读写、同步和调度开销。本表只列主要耗时项, 不再列 diagnostics、zero chunks 和 skipped percentage。

## 9.2 Energy 数据读取与计算细分

| grid             | case      | primary solve s | source compute s | source I/O s | copy in s | recurrence s | copy out s |
|------------------|-----------|-----------------|------------------|--------------|-----------|--------------|------------|
| 8x8-7x7-Nx1000   | water 100 | 18.729          | 3.928            | 5.880        | 17.054    | 12.156       | 53.532     |
| 8x8-7x7-Nx1000   | water 230 | 33.740          | 7.578            | 10.961       | 32.065    | 20.835       | 98.567     |
| 8x8-7x7-Nx1000   | bone 100  | 18.714          | 3.913            | 5.850        | 16.991    | 12.237       | 53.564     |
| 8x8-7x7-Nx1000   | bone 230  | 34.469          | 6.685            | 11.006       | 32.273    | 21.868       | 98.950     |
| 12x12-9x9-Nx2000 | water 100 | 93.827          | 39.764           | 39.616       | 117.177   | 24.789       | 310.327    |
| 12x12-9x9-Nx2000 | water 230 | 172.648         | 67.360           | 74.088       | 219.821   | 41.924       | 571.214    |
| 12x12-9x9-Nx2000 | bone 100  | 93.577          | 34.502           | 39.432       | 116.737   | 25.269       | 305.128    |
| 12x12-9x9-Nx2000 | bone 230  | 173.180         | 59.552           | 73.952       | 219.314   | 43.561       | 564.237    |

`primary solve` 表示 primary proton 在 Eq.15 energy step 中按 streamed chunks 完成的能量更新求解时间。`source compute` 是根据 primary 分布计算 catastrophic scattering 二次粒子源项的 GPU 时间。`source I/O` 是二次源项计算所需 primary 数据在 host/GPU 与 backing store 之间搬运和暂存的时间。`copy in` 包含当前 energy/lane chunk 从 backing store 读入 pinned host buffer 以及 host-to-device copy。`recurrence` 是 Eq.15 energy recurrence kernel 本身的算术计算时间。`copy out` 包含 device-to-host copy、IDD reduction 相关同步, 以及更新后 chunk 写回 backing store。

## 9.3 IDD 曲线图

论文 Section 4.2 将 IDD 定义为横向剂量积分

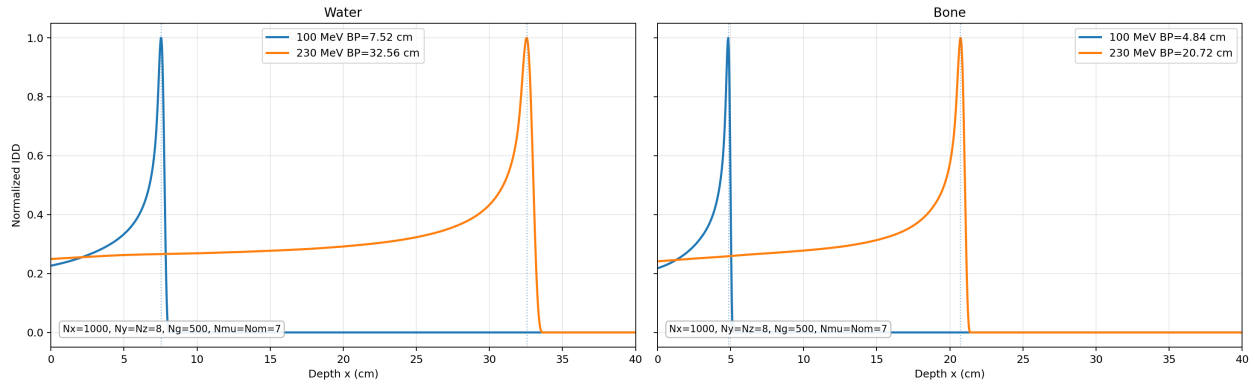
$$\text{IDD}(x) = \iint D(x, y, z) dy dz,$$

Figure 3 画 normalized IDD; 这里同样按每条曲线自身峰值归一化。当前 solver 输出的深度坐标为 time/depth step 终点  $(i+1)\Delta x$ , 因此 8x8-7x7-Nx1000 的第一个样本是  $x = 0.04 \text{ cm}$ , 12x12-9x9-Nx2000

的第一个样本是  $x = 0.02 \text{ cm}$ 。这个坐标约定会给直接用样本最大值报告的 BP 带来  $O(\Delta x)$  的位置偏移。

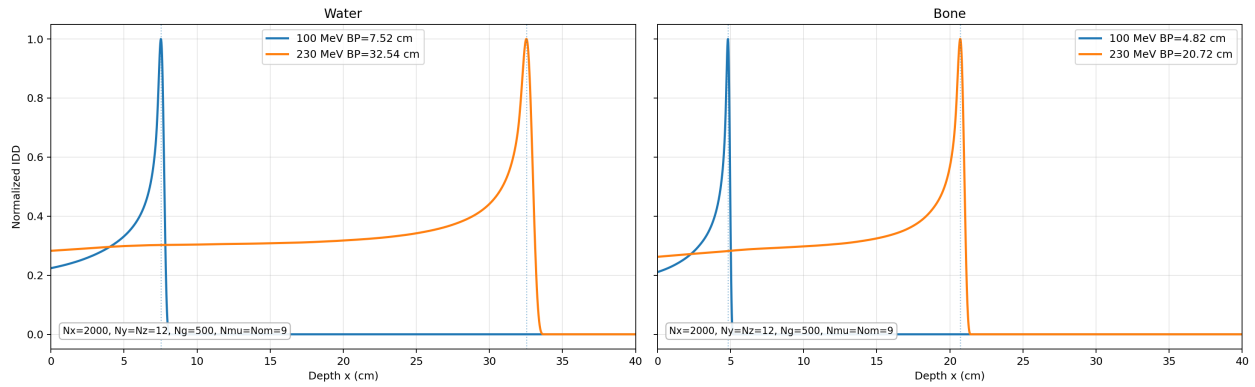
### 8x8-7x7-Nx1000

Eq. 15 IDD on reverted 8x8-7x7-Nx1000 grid



### 12x12-9x9-Nx2000

Eq. 15 IDD on reverted 12x12-9x9-Nx2000 grid



入口归一化值如下：8x8 网格中 water 100/230 MeV 为 0.226962/0.249458，bone 100/230 MeV 为 0.218397/0.241396；12x12 网格中 water 100/230 MeV 为 0.224119/0.282827，bone 100/230 MeV 为 0.210956/0.262506。

#### 9.4 与论文 Table 2 的百分比差异

| grid             | case      | metric | GPU cm  | Table 2 cm | diff cm | rel diff % |
|------------------|-----------|--------|---------|------------|---------|------------|
| 8x8-7x7-Nx1000   | water 100 | BP     | 7.5200  | 7.5600     | -0.0400 | -0.529     |
| 8x8-7x7-Nx1000   | water 100 | P90    | 7.4062  | 7.4180     | -0.0118 | -0.159     |
| 8x8-7x7-Nx1000   | water 100 | D90    | 7.6415  | 7.6710     | -0.0295 | -0.385     |
| 8x8-7x7-Nx1000   | water 100 | D20    | 7.8775  | 7.9450     | -0.0675 | -0.849     |
| 8x8-7x7-Nx1000   | water 230 | BP     | 32.5600 | 32.5400    | +0.0200 | +0.061     |
| 8x8-7x7-Nx1000   | water 230 | P90    | 32.3089 | 32.1720    | +0.1369 | +0.426     |
| 8x8-7x7-Nx1000   | water 230 | D90    | 32.7446 | 32.8280    | -0.0834 | -0.254     |
| 8x8-7x7-Nx1000   | water 230 | D20    | 33.2153 | 33.5130    | -0.2977 | -0.888     |
| 8x8-7x7-Nx1000   | bone 100  | BP     | 4.8400  | 4.7700     | +0.0700 | +1.468     |
| 8x8-7x7-Nx1000   | bone 100  | P90    | 4.7340  | 4.6750     | +0.0590 | +1.261     |
| 8x8-7x7-Nx1000   | bone 100  | D90    | 4.9000  | 4.8370     | +0.0630 | +1.302     |
| 8x8-7x7-Nx1000   | bone 100  | D20    | 5.0561  | 5.0050     | +0.0511 | +1.021     |
| 8x8-7x7-Nx1000   | bone 230  | BP     | 20.7200 | 20.3900    | +0.3300 | +1.618     |
| 8x8-7x7-Nx1000   | bone 230  | P90    | 20.5665 | 20.1680    | +0.3985 | +1.976     |
| 8x8-7x7-Nx1000   | bone 230  | D90    | 20.8612 | 20.5850    | +0.2762 | +1.342     |
| 8x8-7x7-Nx1000   | bone 230  | D20    | 21.1565 | 21.0230    | +0.1335 | +0.635     |
| 12x12-9x9-Nx2000 | water 100 | BP     | 7.5200  | 7.5600     | -0.0400 | -0.529     |
| 12x12-9x9-Nx2000 | water 100 | P90    | 7.4040  | 7.4180     | -0.0140 | -0.189     |
| 12x12-9x9-Nx2000 | water 100 | D90    | 7.6244  | 7.6710     | -0.0466 | -0.607     |
| 12x12-9x9-Nx2000 | water 100 | D20    | 7.8689  | 7.9450     | -0.0761 | -0.958     |
| 12x12-9x9-Nx2000 | water 230 | BP     | 32.5400 | 32.5400    | +0.0000 | +0.000     |
| 12x12-9x9-Nx2000 | water 230 | P90    | 32.2981 | 32.1720    | +0.1261 | +0.392     |
| 12x12-9x9-Nx2000 | water 230 | D90    | 32.7266 | 32.8280    | -0.1014 | -0.309     |
| 12x12-9x9-Nx2000 | water 230 | D20    | 33.2052 | 33.5130    | -0.3078 | -0.918     |
| 12x12-9x9-Nx2000 | bone 100  | BP     | 4.8200  | 4.7700     | +0.0500 | +1.048     |
| 12x12-9x9-Nx2000 | bone 100  | P90    | 4.7360  | 4.6750     | +0.0610 | +1.304     |
| 12x12-9x9-Nx2000 | bone 100  | D90    | 4.8809  | 4.8370     | +0.0439 | +0.909     |
| 12x12-9x9-Nx2000 | bone 100  | D20    | 5.0344  | 5.0050     | +0.0294 | +0.587     |
| 12x12-9x9-Nx2000 | bone 230  | BP     | 20.7200 | 20.3900    | +0.3300 | +1.618     |
| 12x12-9x9-Nx2000 | bone 230  | P90    | 20.5581 | 20.1680    | +0.3901 | +1.934     |
| 12x12-9x9-Nx2000 | bone 230  | D90    | 20.8356 | 20.5850    | +0.2506 | +1.217     |
| 12x12-9x9-Nx2000 | bone 230  | D20    | 21.1376 | 21.0230    | +0.1146 | +0.545     |

### 9.5 两个网格的指标差异

| case      | metric | 12x12 cm | 8x8 cm  | 12x12-8x8 cm | rel change % |
|-----------|--------|----------|---------|--------------|--------------|
| water 100 | BP     | 7.5200   | 7.5200  | +0.0000      | +0.000       |
| water 100 | P90    | 7.4040   | 7.4062  | -0.0022      | -0.030       |
| water 100 | D90    | 7.6244   | 7.6415  | -0.0171      | -0.224       |
| water 100 | D20    | 7.8689   | 7.8775  | -0.0086      | -0.109       |
| water 230 | BP     | 32.5400  | 32.5600 | -0.0200      | -0.061       |
| water 230 | P90    | 32.2981  | 32.3089 | -0.0108      | -0.033       |
| water 230 | D90    | 32.7266  | 32.7446 | -0.0180      | -0.055       |
| water 230 | D20    | 33.2052  | 33.2153 | -0.0101      | -0.030       |
| bone 100  | BP     | 4.8200   | 4.8400  | -0.0200      | -0.413       |
| bone 100  | P90    | 4.7360   | 4.7340  | +0.0020      | +0.042       |
| bone 100  | D90    | 4.8809   | 4.9000  | -0.0191      | -0.390       |
| bone 100  | D20    | 5.0344   | 5.0561  | -0.0217      | -0.429       |
| bone 230  | BP     | 20.7200  | 20.7200 | +0.0000      | +0.000       |
| bone 230  | P90    | 20.5581  | 20.5665 | -0.0084      | -0.041       |
| bone 230  | D90    | 20.8356  | 20.8612 | -0.0256      | -0.123       |
| bone 230  | D20    | 21.1376  | 21.1565 | -0.0189      | -0.089       |

这些差异同时包含深度步长变化、横向/角向离散变化,以及输出坐标约定变化。尤其是 8x8-7x7-Nx1000 的  $\Delta x = 0.04\text{cm}$ , 12x12-9x9-Nx2000 的  $\Delta x = 0.02\text{cm}$ ; 若改用 cell-center 坐标而不是 step-end 坐标,直接由样本最大值读出的 BP 会整体移动约  $\Delta x/2$ 。